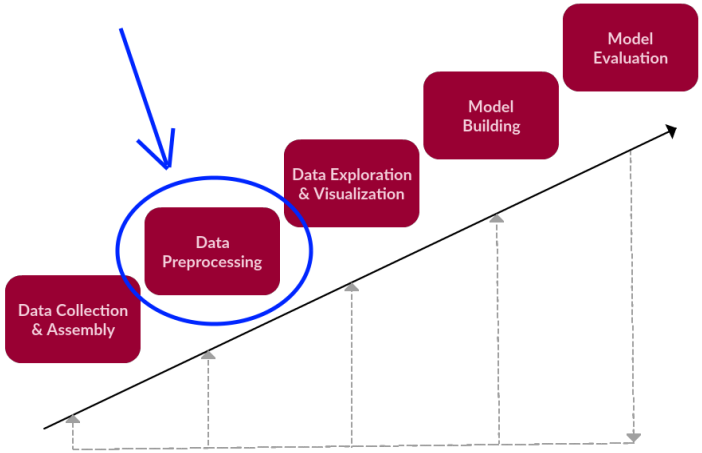
Facultad de Ingeniería de la Universidad Nacional de Asunción

Inteligencia Artificial
2026



Diego Stalder

Christian Torres

Contenido

Preprocesamiento

Manejo de Datos Faltantes:

- Identificación y manejo de valores nulos.
- Eliminación o imputación de datos faltantes.

Limpieza de Datos:

- Detección y manejo de valores atípicos.
- Conversión de tipos de datos según sea necesario.

Codificación de Variables Categóricas:

- Transformación de variables categóricas en variables numéricas.
- Uso de técnicas como One-Hot Encoding.

Normalización y Escalado:

- Normalización de datos numéricos.
- Escalado de datos para algoritmos sensibles a la escala.

Manipulación de Fechas y Tiempos:

- Trabajo con datos de fecha y hora.
- Extracción de características temporales

pip install -U pandas

Python For Data Science *Cheat Sheet*

Pandas Basics

Learn Python for Data Science Interactively at www.DataCamp.com

Pandas

The Pandas library is built on NumPy and provides easy-to-use data structures and data analysis tools for the Python programming language.

Use the following import convention:

```
>>> import pandas as pd
```

Pandas Data Structures

Series

A one-dimensional labeled array capable of holding any data type

Index

A	3
B	-5
C	7
D	4

```
>>> s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

DataFrame

Columns

Index

	Country	Capital	Population
1	Belgium	Brussels	11190846
2	India	New Delhi	1303171035
3	Brazil	Brasilia	207847528

A two-dimensional labeled data structure with columns of potentially different types

```
>>> data = {'Country': ['Belgium', 'India', 'Brazil'],
            'Capital': ['Brussels', 'New Delhi', 'Brasilia'],
            'Population': [11190846, 1303171035, 207847528]}

>>> df = pd.DataFrame(data,
                      columns=['Country', 'Capital', 'Population'])
```

Asking For Help

```
>>> help(pd.Series.loc)
```

Selection

Also see NumPy Arrays

Getting

```
>>> s['b']
 -5

>>> df[1:]
   Country    Capital  Population
1    India  New Delhi  1303171035
2   Brazil   Brasilia   207847528
```

Get one element

Get subset of a DataFrame

Selecting, Boolean Indexing & Setting

By Position

```
>>> df.iloc[[0],[0]]
'Belgium'
>>> df.iat[[0],[0]]
'Belgium'
```

Select single value by row & column

By Label

```
>>> df.loc[[0], ['Country']]
'Belgium'
>>> df.at[[0], ['Country']]
'Belgium'
```

Select single value by row & column labels

By Label/Position

```
>>> df.ix[2]
Country      Brazil
Capital    Brasilia
Population  207847528
>>> df.ix[:,'Capital']
0     Brussels
1    New Delhi
2     Brasilia
>>> df.ix[1,'Capital']
'New Delhi'
```

Select single row of subset of rows

Select a single column of subset of columns

Select rows and columns

Boolean Indexing

```
>>> s[~(s > 1)]
>>> s[(s < -1) | (s > 2)]
>>> df[df['Population']>1200000000]
```

Series s where value is not >1

s where value is <-1 or >2

Use filter to adjust DataFrame

Setting

```
>>> s['a'] = 6
```

Set index a of Series s to 6

Dropping

```
>>> s.drop(['a', 'c'])
>>> df.drop('Country', axis=1)
```

Drop values from rows (axis=0)

Drop values from columns(axis=1)

Sort & Rank

```
>>> df.sort_index(by='Country')
>>> s.order()
>>> df.rank()
```

Sort by row or column index

Sort a series by its values

Assign ranks to entries

Retrieving Series/DataFrame Information

Basic Information

```
>>> df.shape
>>> df.index
>>> df.columns
>>> df.info()
>>> df.count()
```

(rows,columns)

Describe index

Describe DataFrame columns

Info on DataFrame

Number of non-NA values

Summary

```
>>> df.sum()
>>> df.cumsum()
>>> df.min()/df.max()
>>> df.idmin()/df.idmax()
>>> df.describe()
>>> df.mean()
>>> df.median()
```

Sum of values

Cummulative sum of values

Minimum/maximum values

Minimum/Maximum index value

Summary statistics

Mean of values

Median of values

Applying Functions

```
>>> f = lambda x: x*2
>>> df.apply(f)
>>> df.applymap(f)
```

Apply function

Apply function element-wise

Data Alignment

Internal Data Alignment

NA values are introduced in the indices that don't overlap:

```
>>> s3 = pd.Series([7, -2, 3], index=['a', 'c', 'd'])
>>> s + s3
  a     10.0
  b      NaN
  c      5.0
  d      7.0
```

Arithmetic Operations with Fill Methods

You can also do the internal data alignment yourself with the help of the fill methods:

```
>>> s.add(s3, fill_value=0)
  a    10.0
  b    -5.0
  c    5.0
  d    7.0
>>> s.sub(s3, fill_value=2)
>>> s.div(s3, fill_value=4)
>>> s.mul(s3, fill_value=3)
```

I/O

Read and Write to CSV

```
>>> pd.read_csv('file.csv', header=None, nrows=5)
>>> pd.to_csv('myDataFrame.csv')
```

Read and Write to Excel

```
>>> pd.read_excel('file.xlsx')
>>> pd.to_excel('dir/myDataFrame.xlsx', sheet_name='Sheet1')
```

Read multiple sheets from the same file

```
>>> xlsx = pd.ExcelFile('file.xls')
>>> df = pd.read_excel(xlsx, 'Sheet1')
```

Read and Write to SQL Query or Database Table

```
>>> from sqlalchemy import create_engine
>>> engine = create_engine('sqlite:///:memory:')
>>> pd.read_sql("SELECT * FROM my_table;", engine)
>>> pd.read_sql_table('my_table', engine)
>>> pd.read_sql_query("SELECT * FROM my_table;", engine)
```

`read_sql()` is a convenience wrapper around `read_sql_table()` and `read_sql_query()`

```
>>> pd.to_sql('myDf', engine)
```

pip install -U scikit-learn

datacamp

Python For Data Science
Scikit-Learn Cheat Sheet

Learn Scikit-Learn online at www.DataCamp.com

Scikit-learn

Scikit-learn is an open source Python library that implements a range of machine learning, preprocessing, cross-validation and visualization algorithms using a unified interface.

A Basic Example

```
>>> from sklearn import neighbors, datasets, preprocessing
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
>>> iris = datasets.load_iris()
>>> X, y = iris.data[:, :2], iris.target
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=33)
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> X_train = scaler.transform(X_train)
>>> X_test = scaler.transform(X_test)
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
>>> knn.fit(X_train, y_train)
>>> y_pred = knn.predict(X_test)
>>> accuracy_score(y_test, y_pred)
```

> Loading The Data

Also see NumPy & Pandas

Your data needs to be numeric and stored as NumPy arrays or SciPy sparse matrices. Other types that are convertible to numeric arrays, such as Pandas DataFrame, are also acceptable.

```
>>> import numpy as np
>>> X = np.random.random((10,5))
>>> y = np.array(['M','M','F','F','M','F','M','M','F','F','F'])
>>> X[X < 0.7] = 0
```

> Training And Test Data

```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(X,
                                                        y,
                                                        random_state=0)
```

> Model Fitting

Supervised learning

```
>>> lr.fit(X, y) #Fit the model to the data
>>> knn.fit(X_train, y_train)
>>> svc.fit(X_train, y_train)
```

Unsupervised Learning

```
>>> k_means.fit(X_train) #Fit the model to the data
>>> pca_model = pca.fit_transform(X_train) #Fit to data, then transform it
```

> Prediction

Supervised Estimators

```
>>> y_pred = svc.predict(np.random.random((2,5))) #Predict labels
>>> y_pred = lr.predict(X_test) #Predict labels
>>> y_pred = knn.predict_proba(X_test) #Estimate probability of a label
```

Unsupervised Estimators

```
>>> y_pred = k_means.predict(X_test) #Predict labels in clustering algos
```

> Preprocessing The Data

Standardization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(X_train)
>>> standardized_X = scaler.transform(X_train)
>>> standardized_X_test = scaler.transform(X_test)
```

Normalization

```
>>> from sklearn.preprocessing import Normalizer
>>> scaler = Normalizer().fit(X_train)
>>> normalized_X = scaler.transform(X_train)
>>> normalized_X_test = scaler.transform(X_test)
```

Binarization

```
>>> from sklearn.preprocessing import Binarizer
>>> binarizer = Binarizer(threshold=0.0).fit(X)
>>> binary_X = binarizer.transform(X)
```

Encoding Categorical Features

```
>>> from sklearn.preprocessing import LabelEncoder
>>> enc = LabelEncoder()
>>> y = enc.fit_transform(y)
```

Imputing Missing Values

```
>>> from sklearn.preprocessing import Imputer
>>> imp = Imputer(missing_values=0, strategy='mean', axis=0)
>>> imp.fit_transform(X_train)
```

Generating Polynomial Features

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly = PolynomialFeatures(5)
>>> poly.fit_transform(X)
```

> Create Your Model

Supervised Learning Estimators

Linear Regression

```
>>> from sklearn.linear_model import LinearRegression
>>> lr = LinearRegression(normalize=True)
```

Support Vector Machines (SVM)

```
>>> from sklearn.svm import SVC
>>> svc = SVC(kernel='linear')
```

Naive Bayes

```
>>> from sklearn.naive_bayes import GaussianNB
>>> gnb = GaussianNB()
```

KNN

```
>>> from sklearn import neighbors
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
```

Unsupervised Learning Estimators

Principal Component Analysis (PCA)

```
>>> from sklearn.decomposition import PCA
>>> pca = PCA(n_components=0.95)
```

K Means

```
>>> from sklearn.cluster import KMeans
>>> k_means = KMeans(n_clusters=3, random_state=0)
```

> Evaluate Your Model's Performance

Classification Metrics

Accuracy Score

```
>>> knn.score(X_test, y_test) #Estimator score method
>>> from sklearn.metrics import accuracy_score #Metric scoring functions
>>> accuracy_score(y_test, y_pred)
```

Classification Report

```
>>> from sklearn.metrics import classification_report #Precision, recall, f1-score and support
>>> print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
>>> from sklearn.metrics import confusion_matrix
>>> print(confusion_matrix(y_test, y_pred))
```

Regression Metrics

Mean Absolute Error

```
>>> from sklearn.metrics import mean_absolute_error
>>> y_true = [3, -0.5, 2]
>>> mean_absolute_error(y_true, y_pred)
```

Mean Squared Error

```
>>> from sklearn.metrics import mean_squared_error
>>> mean_squared_error(y_test, y_pred)
```

R^2 Score

```
>>> from sklearn.metrics import r2_score
>>> r2_score(y_true, y_pred)
```

Clustering Metrics

Adjusted Rand Index

```
>>> from sklearn.metrics import adjusted_rand_score
>>> adjusted_rand_score(y_true, y_pred)
```

Homogeneity

```
>>> from sklearn.metrics import homogeneity_score
>>> homogeneity_score(y_true, y_pred)
```

V-measure

```
>>> from sklearn.metrics import v_measure_score
>>> metrics.v_measure_score(y_true, y_pred)
```

Cross-Validation

```
>>> from sklearn.cross_validation import cross_val_score
>>> print(cross_val_score(knn, X_train, y_train, cv=4))
>>> print(cross_val_score(lr, X, y, cv=2))
```

> Tune Your Model

Grid Search

```
>>> from sklearn.grid_search import GridSearchCV
>>> params = {"n_neighbors": np.arange(1,3),
              "metric": ["euclidean", "cityblock"]}
>>> grid = GridSearchCV(estimator=knn,
                        param_grid=params)
>>> grid.fit(X_train, y_train)
>>> print(grid.best_score_)
>>> print(grid.best_estimator_.n_neighbors)
```

Randomized Parameter Optimization

```
>>> from sklearn.grid_search import RandomizedSearchCV
>>> params = {"n_neighbors": range(1,5), "weights": ["uniform", "distance"]}
>>> rsearch = RandomizedSearchCV(estimator=knn, param_distributions=params,
                                 cv=4, n_iter=8, random_state=5)
>>> rsearch.fit(X_train, y_train)
>>> print(rsearch.best_score_)
```

Learn Data Skills Online at www.DataCamp.com

La importancia del preprocesamiento

- Entrada de basura → Salida de basura
  - Datos Pobres → malas predicciones / modelado
  - Al igual que no puedes aprender bien cuando te enseñan conocimientos incorrectos

¿Cómo puede ayudarnos el procesamiento previo?

Es obligatorio para algunos modelos

Para disminuir sesgos

Identificar datos redundantes e irrelevantes

Reducir la dimensionalidad

Y más...

Principales tareas de preprocesamiento

- Limpieza de datos
  - Manejo de valores perdidos
  - Manejo de valores atípicos
- Imputación
- Discretización de datos
- Transformación de datos (por ejemplo, normalización)
- Codificación
- El preprocesamiento es principalmente un proceso estadístico y tiene varios métodos adicionales
  - Reducción de dimensionalidad
  - Selección de funciones (no necesariamente preprocesamiento)

Tipos de datos

Tipo	Ejemplo
I. Numérico (doble)	Ingreso económico(por ejemplo: 650,34)
II. Numérico (int)	# de niños (p. ej. 4)
III. Booleano	Sexo (por ejemplo, hombre)
IV. Categórico	Colores (por ejemplo, verde)
V. Ordinal	Satisfacción (por ejemplo, complacido)
VI. Otros	Comentarios

Categórico vs booleano

- Categóricos so esencialmente varios booleanos que están agrupados por alguna lógica

- Ejemplo
  - Característica (color): Verde, Azul, rojo

  vs
  - Característica (es verde): Sí No
  - Característica (es azul): Sí No
  - Característica (es rojo): Sí No

A veces convertimos categóricos en booleanos para el aprendizaje automático

¿Qué tipo de datos faltan?

- Perdidos completamente al azar
- No aleatorios
  - Ejemplos:
    - Es posible que las personas no informen ciertos tipos de datos
    - Puede haber un error en el procesamiento de datos.
    - La estacionalidad puede afectar la recopilación de datos

```
import pandas as pd

 data = {
    'A': [1, 2, None, 4, 5],
    'B': [None, 2, 3, 4, None],
   'C': ['X', 'Y', None, 'X', 'Z']
}

df = pd.DataFrame(data)

df.info()
```

```
print("\nValores nulos por columna:") print(df.isnull().sum())
```

Manejo de datos faltantes: eliminándolos

	Price	Country	Reliability	Mileage	Type	Weight	Disp.	HP
~~Hyundai Sonata 4~~	~~9999~~	~~Korea~~	~~NA~~	~~23~~	~~Medium~~	~~2885~~	~~143~~	~~110~~
Mazda 929 V6	23300	Japan	3	21	Medium	3480	180	158
~~Nissan Maxima V6~~	~~17899~~	~~Japan~~	~~5~~	~~22~~	~~NA~~	~~3200~~	~~180~~	~~160~~
Oldsmobile Cutlass Ciera 4	13150	USA	2	21	Medium	2765	151	110
~~Oldsmobile Cutlass Supreme V6~~	~~14495~~	~~NA~~	~~1~~	~~21~~	~~Medium~~	~~3220~~	~~189~~	~~135~~
Toyota Cressida 6	21498	Japan	3	23	Medium	3480	180	190
Buick Le Sabre V6	16145	USA	3	23	Large	3325	231	165
Chevrolet Caprice V8	14525	USA	1	18	Large	3855	305	170
Ford LTD Crown Victoria V8	17257	USA	3	20	Large	3850	302	150
~~Chevrolet Lumina APV V6~~	~~13995~~	~~USA~~	~~NA~~	~~18~~	~~Van~~	~~3195~~	~~151~~	~~110~~
Dodge Grand Caravan V6	15395	USA	5	18	Van	3735	202	150

- Ignorar la caracteristica
  - Ventaja: simple, típicamente **no sesgado**
  - Con: puede ser una característica muy útil
- Ignorar la muestra
  - Ventaja: simple, todas las funciones se mantienen
  - Con: las muestras eliminadas pueden estar sesgadas
  - Desventaja: los datos pueden reducirse

```
# Eliminación de filas con al menos un valor nulo
df_dropna = df.dropna()
print("\nDataFrame después de eliminar filas con valores nulos:")
print(df_dropna)
# Eliminación de columnas con al menos un valor nulo
df_dropna_column = df.dropna(axis=1)
print("\nDataFrame después de eliminar columnas con valores nulos:")
print(df_dropna_column)
```

Imputación de datos

- Estimar los valores faltantes
- Imputación de datos simple: media, mediana, moda

	Price	Country	Reliability	Mileage	Type	Weight	Disp.	HP
Hyundai Sonata 4	9999	Korea	NA	23	Medium	2885	143	110
Mazda 929 V6	23300	Japan	5	21	Medium	3480	180	158
Nissan Maxima V6	17899	Japan	5	22	NA	3200	180	160
Oldsmobile Cutlass Ciera 4	13150	USA	2	21	Medium	2765	151	110
Oldsmobile Cutlass Supreme V6	14495	NA	1	21	Medium	3220	189	135
Toyota Cressida 6	21498	Japan	3	23	Medium	3480	180	190
Buick Le Sabre V6	16145	USA	3	23	Large	3325	231	165
Chevrolet Caprice V8	14525	USA	1	18	Large	3855	305	170
Ford LTD Crown Victoria V8	17257	USA	3	20	Large	3850	302	150
Chevrolet Lumina APV V6	13995	USA	NA	18	Van	3195	151	110
Dodge Grand Caravan V6	15395	USA	3	18	Van	3735	202	150

Media (confiabilidad): (5 + 5 + 2 + 1 + 3 + 3 + 1 + 3 + 3) / 9 = **2,88**
Mediana (confiabilidad): 1 1 2 3 **3** 3 3 5 5
Moda (país): **Estados Unidos = 6**, Japón = 3, Corea = 1.

```
# Imputación utilizando la estrategia de media
df_imputed_mean = df.fillna(df.mean())
print("\nDataFrame después de imputar valores nulos usando la media:")
print(df_imputed_mean )
```

```
df_imputed_median = df.fillna(df.median())
print("\nDataFrame después de imputar valores nulos en columnas numéricas
usando la mediana:")
print(df_imputed_median)
```

Imputación de datos

```
import pandas as pd

 data = {
    'A': [1, 2, None, 4, 5],
    'B': [None, 2, 3, 4, None],
   'C': ['X', 'Y', None, 'X', 'Z']
}

df = pd.DataFrame(data)

df.info()
```

```
# Imputación de valores nulos en columnas categóricas usando la moda
df_imputed_mode = df.apply(lambda col: col.fillna(col.mode()[0]) if col.dtype == 'object' else
col.fillna(col.mean()), axis=0)
print("\nDataFrame después de imputar valores nulos en columnas categóricas usando la moda:")
print(df_imputed_mode)
```

Imputación de datos

sklearn.impute: Impute

Transformers for missing value imputation

User guide: See the Imputation of missing values section for further details.

impute.SimpleImputer(*[, missing_values, ...])	Univariate imputer for completing missing values with simple strategies.
impute.IterativeImputer([estimator, ...])	Multivariate imputer that estimates each feature from all the others.
impute.MissingIndicator(*[, missing_values, ...])	Binary indicators for missing values.
impute.KNNImputer(*[, missing_values, ...])	Imputation for completing missing values using k-Nearest Neighbors.

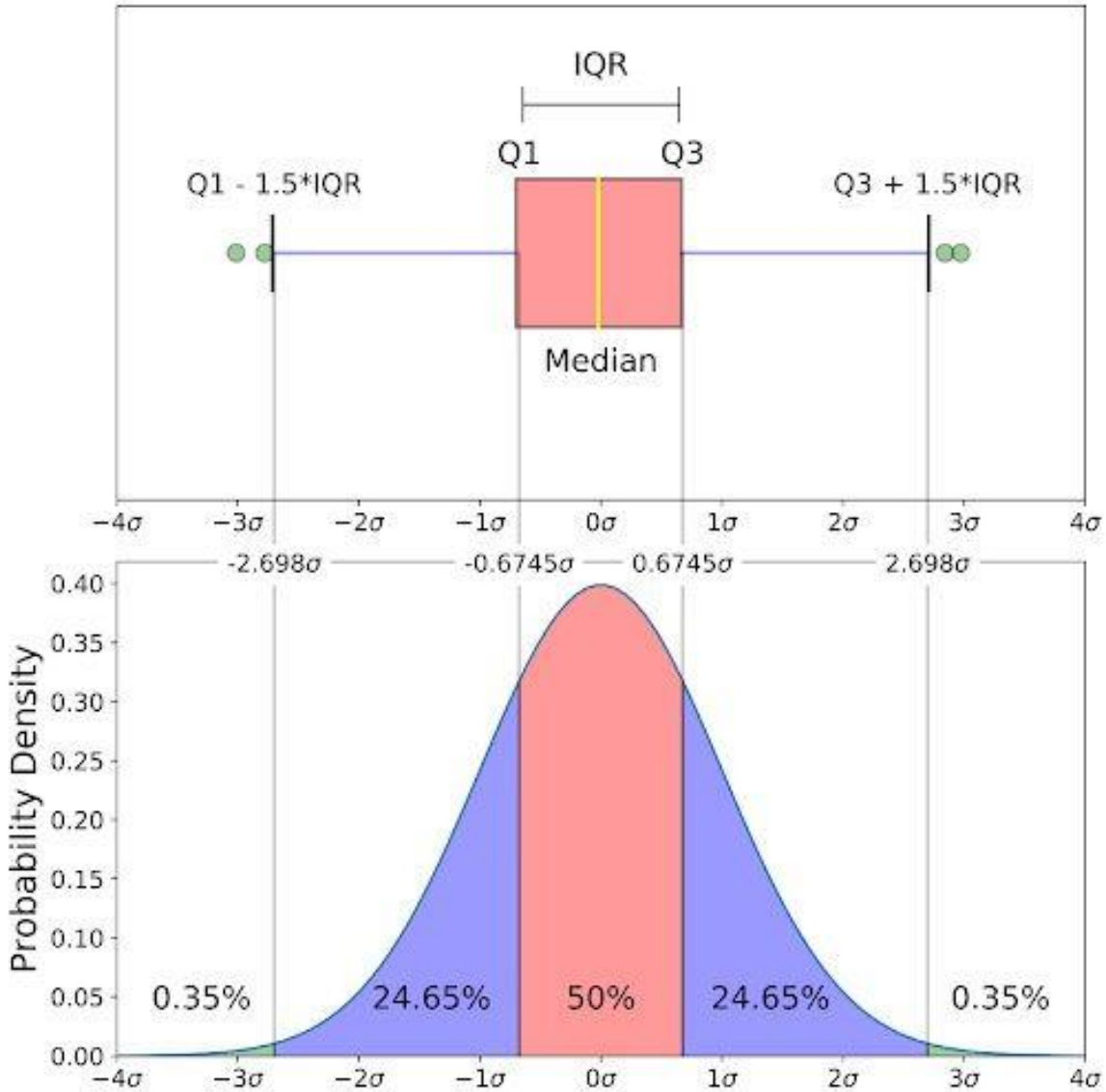
```
from sklearn.impute import SimpleImputer

# Imputación utilizando la estrategia de media
imputer = SimpleImputer(strategy='mean')
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

print("\nDataFrame después de imputar valores nulos usando la media:")
print(df_imputed)

# Imputación utilizando la estrategia de mediana

imputer = SimpleImputer(strategy='median')
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
print("\nDataFrame después de imputar valores nulos usando la mediana:")
print(df_imputed)
```

Valores atípicos: causas y motivos para detectar

- Porque:
  - Error experimental
  - Verdadera variabilidad en la medición
    - Por casualidad
    - Debido a distribuciones sesgadas (no simétricas)
  - Poblaciones mixtas (modeladas por modelos mixtos)

- ¿Por qué detectarlos?
  - Para entender información sobre posibles errores
  - Comprender la verdadera distribución de datos
  - Detección de anomalías en fraudes, seguridad, etc.

Box Plot


IQR
Q1
Q3
Q1 - 1.5*IQR
Q3 + 1.5*IQR
Median
−4σ
−3σ
−2σ
−1σ
0σ
1σ
2σ
3σ
4σ
-2.698σ
-0.6745σ
0.6745σ
2.698σ
Probability Density
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.35%
24.65%
50%
24.65%
0.35%

Manejo de valores atípicos en el aprendizaje automático

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Generamos un conjunto de datos de prueba con valores atípicos
np.random.seed(0)
data = {
    'A': np.random.normal(loc=0, scale=1, size=100),
    'B': np.random.normal(loc=0, scale=1, size=100),
    'C': np.random.normal(loc=0, scale=1, size=100)
}
# Introducimos valores atípicos en la columna 'A'
data['A'][0] = 10
data['A'][1] = -10
df = pd.DataFrame(data)
```

```
# Creamos un boxplot para visualizar la distribución de los datos
df.boxplot()
plt.title('Boxplot de los datos')
plt.show()
```

Manejo de valores atípicos en el aprendizaje automático

```
from scipy import stats
# Calculamos el z-score para cada punto de datos en la columna 'A'
z_scores = stats.zscore(df['A'])

# Mostramos los puntos de datos con un z-score absoluto mayor que 3
outliers = df[abs(z_scores) > 3]
print("Valores atípicos detectados:")
print(outliers)
```

```
# Calculamos el rango intercuartílico (IQR) para cada columna
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1

# Definimos el límite para considerar valores atípicos
outlier_threshold = 1.5

# Identificamos valores atípicos utilizando el criterio del IQR
outliers = (df < (Q1 - outlier_threshold * IQR)) | (df > (Q3 + outlier_threshold * IQR))

print("Valores atípicos detectados:")
print(outliers.sum())
```

Terminología

- Transformación
  - Aplicando una función a cada punto z en los datos: $y_{yo} = F(z_{yo})$

- Escalado

  Conversión de datos a una escala diferente (como Celsius y Fahrenheit). Normalmente lineal: y = ax + b

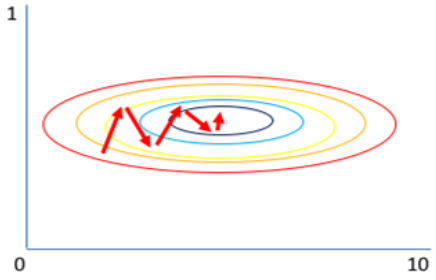
- Normalización
  - Aplicando una transformación para que los datos transformados se distribuyan aproximadamente normalmente
  - O también puede significar poner diferentes variables en una escala común (en este caso es también conocido como escalado)

Razones para normalizar y transformar

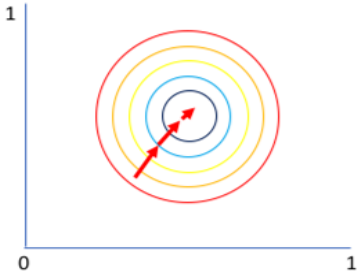
- Fácil comparación de valores
- En algunos algoritmos, las funciones objetivo no funcionarán correctamente (o rápidamente) sin él
- Puede crear características complejas que pueden mejorar el modelo (o hacerlo no lineal)

```
data = {
    'A': np.random.randint(0, 100, 10),
    'B': np.random.randint(0, 1000, 10),
    'C': np.random.randint(0, 10000, 10)
}
df = pd.DataFrame(data)
```

Why normalize?



Gradient of larger parameter dominates the update

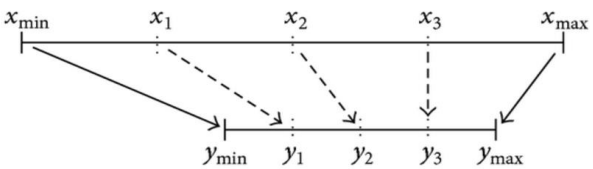


Both parameters can be updated in equal proportions

Normalización Min-Max

- Normalización mínima / máxima a [0,1]

$$X_{i,\,0\text{ to }1} = \frac{X_i - X_{Min}}{X_{Max} - X_{Min}}$$



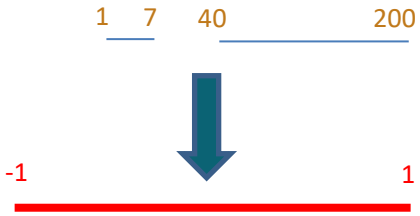

```
# Normalización Min-Max en pandas
df_minmax_pandas = (df - df.min()) / (df.max() - df.min())
print("\nDataFrame después de la normalización Min-Max en pandas:")
print(df_minmax_pandas)
```

```
from sklearn.preprocessing import MinMaxScaler

# Creamos un objeto MinMaxScaler
scaler_minmax = MinMaxScaler()

# Aplicamos la normalización Min-Max a los datos
df_minmax = scaler_minmax.fit_transform(df)
df_minmax = pd.DataFrame(df_minmax, columns=df.columns)
print("\nDataFrame después de la normalización Min-Max:")
print(df_minmax)
```

- Normalización mínima / máxima a [-1,1]

(si queremos que 0 sea el punto central)

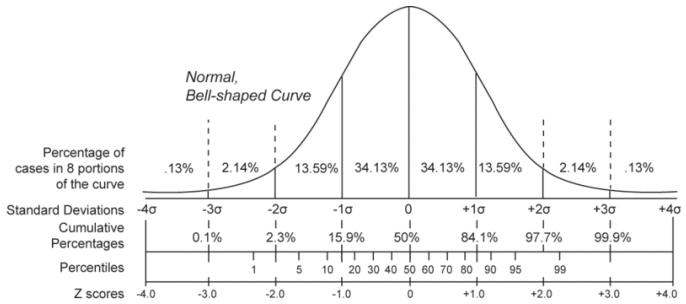
$$X_{i,-1\text{ to }1} = \frac{2X_i - X_{Min} - X_{Max}}{X_{Max} - X_{Min}}$$



Estandarización de datos (normalización Z)

- Restar la media y dividir por la desviación estándar
- La media se convierte en 0, las unidades son sd

$$X_{i,\,1\sigma} = \frac{X_i - \bar{X}_S}{\sigma_{X,\,S}}$$

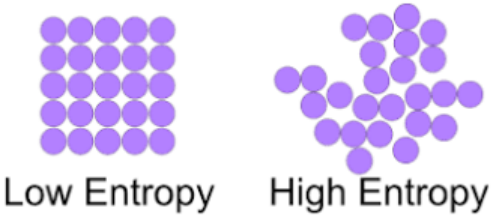



```
# Estandarización Z-score en pandas
df_standard_pandas = (df - df.mean()) / df.std()
print("\nDataFrame después de la estandarización Z-score en pandas:")
print(df_standard_pandas)
```

```
from sklearn.preprocessing import StandardScaler

# Creamos un objeto StandardScaler
scaler_standard = StandardScaler()

# Aplicamos la estandarización Z-score a los datos
df_standard = scaler_standard.fit_transform(df)
df_standard = pd.DataFrame(df_standard, columns=df.columns)
print("\nDataFrame después de la estandarización Z-score:")
print(df_standard)
```

Elegir el método de normalización

- Gran variedad de datos: (es decir, $ 4 a $ 120 000 000)
  - La transformación de registros suele ser buena
- Datos sesgados (a menudo un rango grande)
  - La transformación de registros suele ser buena



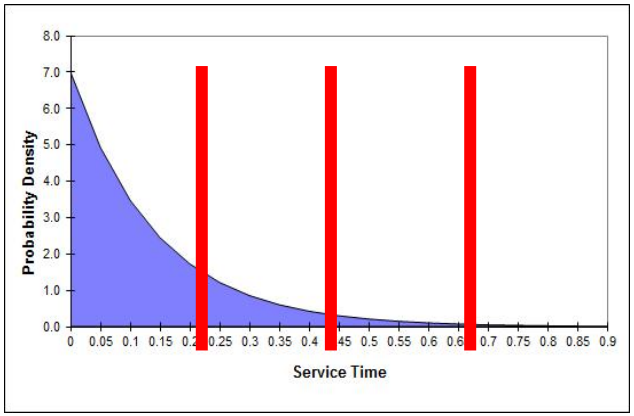
- Si la entropía es alta → normalmente normalizar a [-1,1] es bueno
- Si la entropía es baja → a menudo la estandarización z es buena
- Si está distribuido casi normalmente → la estandarización z es buena

Discretización de variables numéricas

- A dar significado: es decir, bajo vs normal vs alto
- A visualizar datos - histogramas
- Necesario para algunos modelos / medidas:
  - Bayes ingenuo
  - Información mutua
- Mejorar los resultados de varios modelos, incluso si no es necesario

```
import pandas as pd
import numpy as np

# Generamos un conjunto de datos aleatorios bimodal
data_bimodal = np.concatenate([np.random.normal(loc=5, scale=1, size=50),
np.random.normal(loc=15, scale=3, size=50)])

# Creamos un DataFrame con los datos
df = pd.DataFrame({'Variable': data_bimodal})
print("DataFrame con datos bimodales:")
print(df)
```

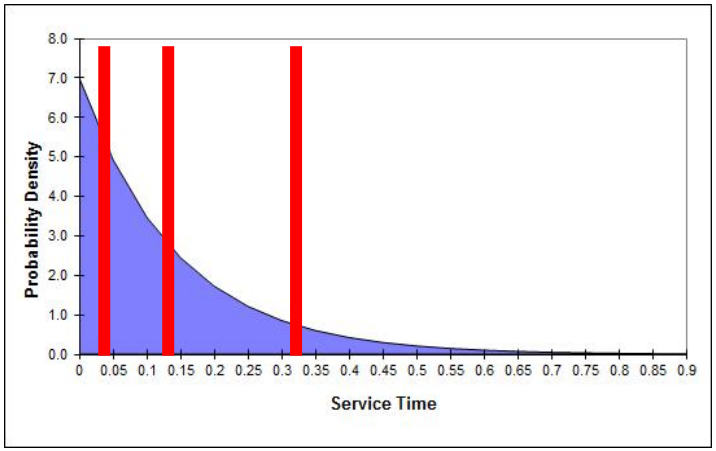
Partición de igual ancho (distancia)



- Se divide en N intervalos de igual tamaño: grilla uniforme
- Ancho de intervalos: W = (valor más alto - valor más bajo) / N.
- Valor del contenedor a menudo es mediana, pero puede ser promedio u otro
- Ventajas: El más sencillo
- Contras: Datos sesgados no se maneja bien, valores atípicos puede dominar

```
# Discretización en bins de igual tamaño
df['Bins_Igual_Tamanho'] = pd.cut(df['Variable'], bins=4, labels=False)
print("\nDataFrame después de la discretización en bins de igual tamaño:")
print(df)
```

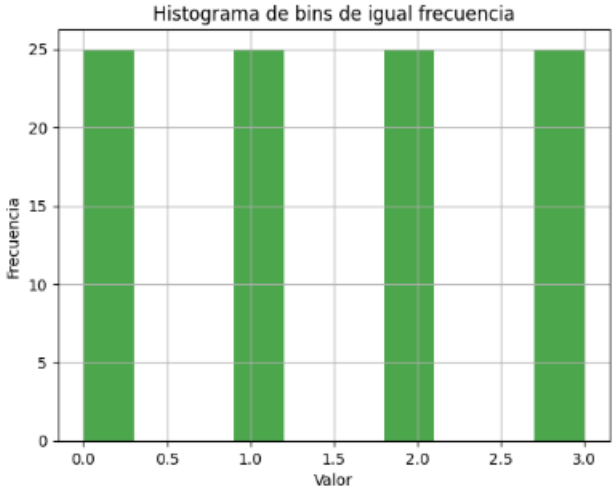
Partición de igual profundidad (frecuencia)

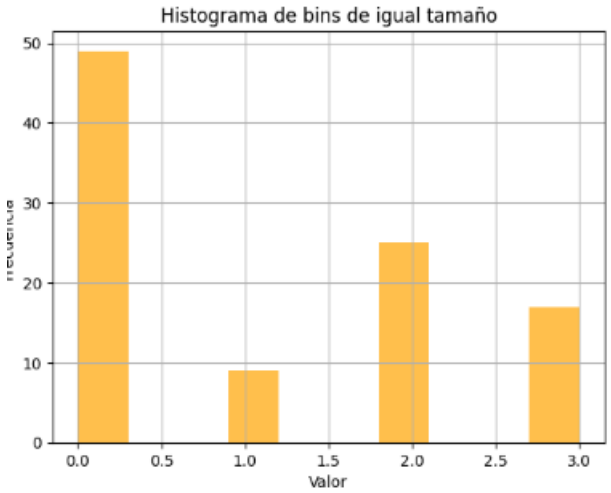


- Divide el rango en N intervalos, cada uno con aproximadamente el mismo número de muestras
- Buen escalado de datos
- Administrar atributos categóricos puede ser complicado

```
# Discretización en bins de igual frecuencia
df['Bins_Igual_Frecuencia'] = pd.qcut(df['Variable'], q=4, labels=False)
print("\nDataFrame después de la discretización en bins de igual frecuencia:")print(df)
```

Histogramas

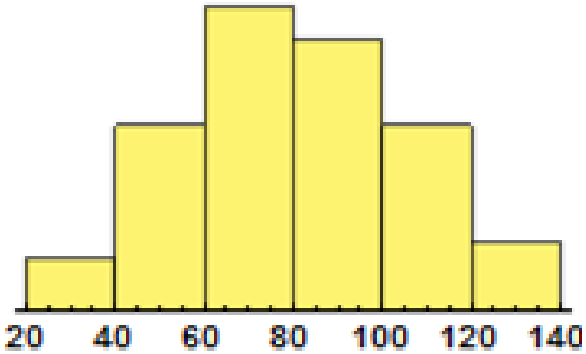


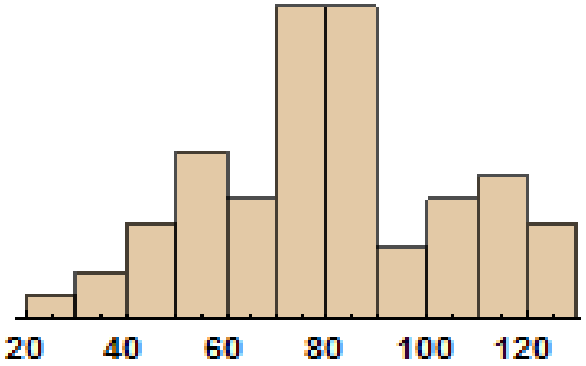



```
# Graficar el histograma de los bins de igual frecuencia
plt.hist(df['Bins_Igual_Frecuencia'],color='green', alpha=0.7)
plt.title('Histograma de bins de igual frecuencia')plt.xlabel('Valor’)
plt.ylabel('Frecuencia’)
plt.grid(True)
plt.show()
# Graficar el histograma de los bins de igual tamaño
plt.hist(df['Bins_Igual_Tamanho'], color='orange', alpha=0.7)
plt.title('Histograma de bins de igual tamaño’)
plt.xlabel('Valor’)
plt.ylabel('Frecuencia’)
plt.grid(True)
plt.show()
```

Seleccionar el número de bin

- Error de discretización
- Puede hacer que los datos se vean de manera diferente cuando los tamaños de muestra son pequeños





Mismos datos, diferente número de contenedores

```
# Graficar el histograma de los bins de igual frecuencia
plt.hist(df['Bins_Igual_Frecuencia'],color='green', alpha=0.7)
plt.title('Histograma de bins de igual frecuencia')plt.xlabel('Valor’)
plt.ylabel('Frecuencia’)
plt.grid(True)
plt.show()
# Graficar el histograma de los bins de igual tamaño
plt.hist(df['Bins_Igual_Tamanho'], color='orange', alpha=0.7)
plt.title('Histograma de bins de igual tamaño’)
plt.xlabel('Valor’)
plt.ylabel('Frecuencia’)
plt.grid(True)
plt.show()
```

Seleccionar el número de bin

```
import matplotlib.pyplot as plt
# Graficar el histograma de la variable original
plt.hist(df['Variable'], bins='auto', color='blue', alpha=0.7)
plt.title('Histograma de la variable original’)
plt.xlabel('Valor')plt.ylabel('Frecuencia’)
plt.grid(True)plt.show()
```

En este histograma, hemos utilizado bins='auto', lo que significa que matplotlib determinará automáticamente el número de bins utilizando un algoritmo optimizado.

```
# Graficar el histograma de los bins de igual frecuencia
plt.hist(df['Bins_Igual_Frecuencia'],color='green', alpha=0.7)
plt.title('Histograma de bins de igual frecuencia')plt.xlabel('Valor’)
plt.ylabel('Frecuencia’)
plt.grid(True)
plt.show()
# Graficar el histograma de los bins de igual tamaño
plt.hist(df['Bins_Igual_Tamanho'], color='orange', alpha=0.7)
plt.title('Histograma de bins de igual tamaño’)
plt.xlabel('Valor’)
plt.ylabel('Frecuencia’)
plt.grid(True)
plt.show()
```

Codificar las variables categóricas

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

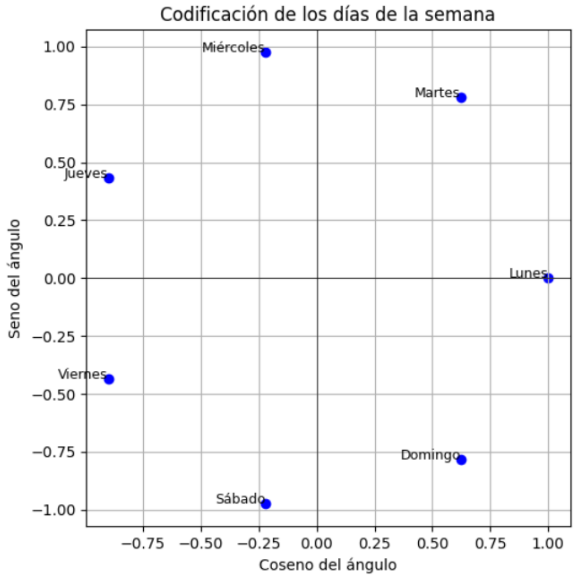
Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

```
import pandas as pd
# Creamos un DataFrame de ejemplo con variables categóricas
data = {'Color': ['Rojo', 'Verde', 'Azul', 'Rojo', 'Azul'],
'Tamaño': ['Pequeño', 'Grande', 'Mediano', 'Grande', 'Pequeño’]}
df = pd.DataFrame(data)print("DataFrame original:")
```

```
print(df)# Aplicamos One-Hot Encoding utilizando get_dummies de pandas
df_encoded = pd.get_dummies(df)
print("\nDataFrame después de One-Hot Encoding:")print(df_encoded)
```

```
from sklearn.preprocessing import OneHotEncoder
# Creamos un codificador OneHotEncoder
encoder = OneHotEncoder(sparse=False)

# Ajustamos y transformamos los datos
encoded_data = encoder.fit_transform(df)
encoded_df = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(['Color',
'Tamaño']))
print("\nDataFrame después de One-Hot Encoding:")
print(encoded_df)
```

Codificar las variables categóricas




```
# Creamos un DataFrame de ejemplo con los días de la semana
dias_semana = ['Lunes', 'Martes', 'Miércoles', 'Jueves',
'Viernes', 'Sábado', 'Domingo']

# Creamos un DataFrame con una columna de días de la semana
df = pd.DataFrame({'Dia': dias_semana})
print("DataFrame original:")
print(df)
```

```
# Creamos una función para codificar los días de la semana usando seno y coseno
def codificar_dia(dia):
    angulo = dias_semana.index(dia) * (2 * np.pi / len(dias_semana))
    return np.sin(angulo), np.cos(angulo)
```

```
# Aplicamos la función a cada fila del DataFrame
df[['Seno_Dia', 'Coseno_Dia']] = df['Dia'].apply(codificar_dia).apply(pd.Series)

print("\nDataFrame después de codificar los días de la semana usando seno y coseno:")
print(df)
```